

Implementasi Algoritma Cosine Similarity

LMS SMAN 1

Lokasi file utama: app/Services/CosineSimilarityService.php

Controller pemanggil: app/Http/Controllers/Guru/SimilarityController.php

Ringkasan alur:

1. preprocess(): teks dirapikan dulu, tanda baca dihapus, lalu dipecah jadi kata-kata.
2. Kata umum dan kata yang terlalu pendek dibuang agar hasil perbandingan lebih fokus.
3. buildTFVector(): setiap kata dihitung frekuensinya lalu dibuat menjadi vektor TF.
4. calculateSimilarity(): dua vektor dibandingkan menggunakan rumus cosine similarity.
5. compareAnswers(): semua pasangan jawaban dalam satu tugas dibandingkan dan hasilnya disimpan.

Rumus:

$$\text{similarity} = \text{dot}(A, B) / (|A| * |B|)$$

Hasilnya diubah ke persen dengan round(similarity * 100, 2).

Status hasil:

- > 70% : plagiat
- 30-70% : warning
- < 30% : safe

Kode sumber:

```
1: <?php
2:
3: namespace App\Services;
4:
5: use App\Models\JawabanTugas;
6: use App\Models\SimilarityResult;
7: use App\Models\Tugas;
8:
9: class CosineSimilarityService
10: {
11:     /**
12:      * Daftar kata umum yang tidak terlalu berpengaruh saat membandingkan jawaban
13:      */
14:     private array $stopwords = [
15:         'dan', 'di', 'ke', 'dari', 'yang', 'untuk', 'pada', 'dengan', 'adalah',
16:         'ini', 'itu', 'atau', 'juga', 'sudah', 'saya', 'anda', 'kami', 'kita',
17:         'mereka', 'dia', 'akan', 'bisa', 'ada', 'tidak', 'bukan', 'hanya',
18:         'lebih', 'sangat', 'telah', 'sedang', 'masih', 'belum', 'harus',
19:         'dapat', 'oleh', 'karena', 'jika', 'maka', 'saat', 'ketika',
20:         'setelah', 'sebelum', 'antara', 'dalam', 'luar', 'atas', 'bawah',
21:         'seperti', 'sama', 'lain', 'semua', 'setiap', 'para', 'tersebut',
22:         'yaitu', 'bahwa', 'tentang', 'tanpa', 'melalui', 'secara', 'serta',
23:         'namun', 'tetapi', 'melainkan', 'selain', 'maupun', 'agar', 'supaya',
24:         'apabila', 'adapun', 'demikian', 'begitu', 'sehingga', 'meskipun',
25:         'walaupun', 'biarpun', 'sejak', 'sampai', 'hingga', 'terhadap',
26:         'mengenai', 'perihal', 'yakni', 'sebagai', 'merupakan',
27:         'the', 'a', 'an', 'is', 'are', 'was', 'were', 'be', 'been', 'being',
28:         'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would', 'could',
29:         'should', 'may', 'might', 'shall', 'can', 'of', 'in', 'to', 'for',
30:         'with', 'on', 'at', 'by', 'from', 'as', 'into', 'through', 'during',
31:         'before', 'after', 'above', 'below', 'between', 'under', 'again',
32:         'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why',
33:         'how', 'all', 'each', 'every', 'both', 'few', 'more', 'most', 'other',
34:         'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so',
35:         'than', 'too', 'very', 'just', 'because', 'but', 'and', 'if', 'or',
36:         'while', 'about', 'against', 'it', 'this', 'that', 'these', 'those',
37:     ];
38:
39:     /**
40:      * Tahap awal untuk merapikan teks jawaban
41:      * - ubah jadi huruf kecil
42:      * - hapus tanda baca
43:      * - pecah teks jadi kata-kata
44:      * - buang kata umum
45:      */
```

```

46: public function preprocess(string $text): array
47: {
48:     // Ubah semua huruf jadi kecil biar perbandingannya konsisten
49:     $text = mb_strtolower($text, 'UTF-8');
50:
51:     // Hapus tanda baca dan karakter khusus, sisakan huruf, angka, dan spasi
52:     $text = preg_replace('/[^\p{L}\p{N}\s]/u', ' ', $text);
53:
54:     // Rapikan spasi yang berlebihan
55:     $text = preg_replace('/\s+/', ' ', trim($text));
56:
57:     // Pecah teks jadi kumpulan kata
58:     $tokens = explode(' ', $text);
59:
60:     // Buang kata umum dan kata yang terlalu pendek
61:     $tokens = array_filter($tokens, function ($token) {
62:         return strlen($token) >= 2 && !in_array($token, $this->stopwords);
63:     });
64:
65:     return array_values($tokens);
66: }
67:
68: /**
69:  * Membuat vektor Term Frequency (TF) dari kata-kata yang sudah dirapikan
70:  */
71: public function buildTFVector(array $tokens): array
72: {
73:     $tf = [];
74:     $totalTokens = count($tokens);
75:
76:     if ($totalTokens === 0) {
77:         return $tf;
78:     }
79:
80:     // Hitung berapa kali tiap kata muncul
81:     $frequency = array_count_values($tokens);
82:
83:     // Bagi dengan total kata agar nilainya lebih seimbang
84:     foreach ($frequency as $term => $count) {
85:         $tf[$term] = $count / $totalTokens;
86:     }
87:
88:     return $tf;
89: }
90:
91: /**
92:  * Menghitung kemiripan dua jawaban dengan Cosine Similarity
93:  * Hasil akhirnya berupa persentase 0 sampai 100
94:  */
95: public function calculateSimilarity(array $vec1, array $vec2): float
96: {
97:     if (empty($vec1) || empty($vec2)) {
98:         return 0.0;
99:     }
100:
101:     // Gabungkan semua kata unik dari dua jawaban
102:     $allTerms = array_unique(array_merge(array_keys($vec1), array_keys($vec2)));
103:
104:     // Hitung dot product dan panjang masing-masing vektor
105:     $dotProduct = 0.0;
106:     $magnitude1 = 0.0;
107:     $magnitude2 = 0.0;
108:
109:     foreach ($allTerms as $term) {
110:         $val1 = $vec1[$term] ?? 0.0;
111:         $val2 = $vec2[$term] ?? 0.0;
112:
113:         $dotProduct += $val1 * $val2;
114:         $magnitude1 += $val1 * $val1;
115:         $magnitude2 += $val2 * $val2;
116:     }
117:
118:     $magnitude1 = sqrt($magnitude1);
119:     $magnitude2 = sqrt($magnitude2);
120:
121:     // Kalau salah satu vektor kosong, hasilnya dianggap 0
122:     if ($magnitude1 == 0 || $magnitude2 == 0) {
123:         return 0.0;
124:     }

```

```

125:
126:     // Rumus cosine similarity: dot(A,B) / (|A| * |B|)
127:     $similarity = $dotProduct / ($magnitude1 * $magnitude2);
128:
129:     // Ubah hasilnya ke bentuk persen
130:     return round($similarity * 100, 2);
131: }
132:
133: /**
134:  * Membandingkan semua jawaban dalam satu tugas yang sama
135:  */
136: public function compareAnswers(int $tugasId): array
137: {
138:     $tugas = Tugas::findOrFail($tugasId);
139:
140:     // Ambil jawaban teks dari tugas ini saja
141:     $jawaban = JawabanTugas::where('tugas_id', $tugasId)
142:         ->whereNotNull('jawaban_text')
143:         ->where('jawaban_text', '!=', '')
144:         ->with('siswa')
145:         ->get();
146:
147:     if ($jawaban->count() < 2) {
148:         return [
149:             'message' => 'Minimal 2 jawaban diperlukan untuk perbandingan.',
150:             'results' => [],
151:         ];
152:     }
153:
154:     // Hapus hasil lama supaya data tidak dobel saat dicek ulang
155:     SimilarityResult::where('tugas_id', $tugasId)->delete();
156:
157:     // Rapikan teks lalu buat vektor untuk tiap jawaban
158:     $vectors = [];
159:     foreach ($jawaban as $j) {
160:         $tokens = $this->preprocess($j->jawaban_text);
161:         $vectors[$j->id] = $this->buildTFVector($tokens);
162:     }
163:
164:     $results = [];
165:     $jawabanArray = $jawaban->values();
166:
167:     // Bandingkan setiap pasangan jawaban satu per satu
168:     for ($i = 0; $i < $jawabanArray->count(); $i++) {
169:         for ($k = $i + 1; $k < $jawabanArray->count(); $k++) {
170:             $j1 = $jawabanArray[$i];
171:             $j2 = $jawabanArray[$k];
172:
173:             $percentage = $this->calculateSimilarity(
174:                 $vectors[$j1->id],
175:                 $vectors[$j2->id]
176:             );
177:
178:             // Tentukan status dari nilai kemiripannya
179:             $status = 'safe';
180:             if ($percentage > 70) {
181:                 $status = 'plagiat';
182:             } elseif ($percentage >= 30) {
183:                 $status = 'warning';
184:             }
185:
186:             // Simpan hasil pengecekan ke database
187:             $result = SimilarityResult::create([
188:                 'tugas_id' => $tugasId,
189:                 'jawaban_1_id' => $j1->id,
190:                 'jawaban_2_id' => $j2->id,
191:                 'similarity_percentage' => $percentage,
192:                 'status' => $status,
193:                 'tahun_ajaran_id' => $tugas->tahun_ajaran_id,
194:             ]);
195:
196:             $results[] = [
197:                 'siswa_1' => $j1->siswa->nama,
198:                 'siswa_2' => $j2->siswa->nama,
199:                 'percentage' => $percentage,
200:                 'status' => $status,
201:             ];
202:         }
203:     }

```

```
204:
205:     // Urutkan dari nilai similarity paling tinggi
206:     usort($results, fn($a, $b) => $b['percentage'] <=> $a['percentage']);
207:
208:     return [
209:         'message' => 'Analisis similarity selesai.',
210:         'total_jawaban' => $jawaban->count(),
211:         'total_comparisons' => count($results),
212:         'results' => $results,
213:     ];
214: }
215: }
```